

PIPELINE _ Klebsiellitas

Velazquez Zyanya, Contreras Yeimi, Mendoza Marina

BLOQUE 0 — Descarga automatizada de datos y genoma de referencia

0.1 Pipeline_2.0.py (pipeline maestro: descarga SRA + genoma)

Rol en el pipeline

Este es el **primer script** que se debe ejecutar en un proyecto nuevo.

Está diseñado para ser **totalmente reproducible** y **automatizar** todo el proceso de:

- encontrar todos los SRR del BioProject,
- descargarlos,
- convertirlos a FASTQ,
- encontrar y descargar automáticamente el mejor genoma de referencia desde NCBI Assembly.

Flujo interno del script

(1) Consulta de SRR IDs

- Usa `Entrez.esearch(db="sra", term=SRP291413)` para identificar los IDs del proyecto.
- Para cada ID, usa `Entrez.esummary` para extraer la sección `<Runs>`.
- Extrae valores `acc="SRR12998916"`, etc.
- Crea una lista limpia de SRR:

```
['SRR12998916', 'SRR12998917', ..., 'SRR12998921']
```

(2) Descarga de cada SRR con SRA Toolkit

Para cada SRR:

- Comprueba si existe `raw_sra/SRRxxxxxx.sra` para evitar descargas repetidas.

- Ejecuta:

```
prefetch SRRxxxxxx -O raw_sra/
```

(3) Conversión a FASTQ pareado

Para cada SRR:

- Si los FASTQ ya existen, los omite (reproducibilidad).
- Si no, ejecuta:

```
fasterq-dump SRRxxxxxx --split-files -O fastq/
```

Esto genera:

```
SRRxxxxxx_1.fastq  
SRRxxxxxx_2.fastq
```

(4) Descarga del genoma de referencia

Llama a:

```
fetch_reference_package(  
    organism_query="Klebsiella sp. AQSCr",  
    strain="AQSCr",  
    bioproject="PRJNA341863",  
    out_dir="genome/"  
)
```

Este módulo:

- Busca ensamblajes en NCBI Assembly.
- Pondera objetivamente cuál es el mejor.
- Descarga:
 - `_genomic.fna.gz` (genoma)
 - `_genomic.gff.gz` (anotaciones)
- Escribe `ASSEMBLY_PROVENANCE.txt`.

(5) Descompresión automática

El script descomprime los `.gz` :

- `GCF_XXXXX_genomic.fna`
- `GCF_XXXXX_genomic.gff`

Los deja listos para indexar y para featureCounts.

En resumen

`Pipeline_2.0.py` hace:

- ✓ Descarga **automática y reproducible** de datos SRA
 - ✓ Conversión a FASTQ
 - ✓ Descubrimiento inteligente del mejor genoma RefSeq
 - ✓ Descarga + descompresión del genoma
 - ✓ Estructura de proyecto lista para el análisis
-

0.2 `descarga_genoma_referencia.py` (módulo reusable: descarga reproducible del genoma)

Rol

Provee una interfaz robusta, reusable y con trazabilidad para obtener cualquier genoma de referencia desde NCBI Assembly.

Este módulo es independiente y puede usarse en otros proyectos.

Flujo interno

(1) Búsqueda de ensamblajes

Función: `find_best_assembly(...)`

- Construye consultas usando:

```
organism_query  
strain  
bioproject  
latest[filter]
```

Ejemplo:

```
"Klebsiella sp. AQSCr AND latest[filter] AND PRJNA341863[BioProject]"
```

- Recupera hasta 100 ensamblajes candidatos.
- Los evalúa con un sistema de puntuación:

Criterios de evaluación

1. RefSeq_category

- reference genome (3 pts)
- representative genome (2 pts)
- none (0 pts)

2. AssemblyStatus

- Complete Genome (4 pts)
- Chromosome (3 pts)
- Scaffold (2 pts)
- Contig (1 pt)

3. Namespace

- GCF_ (RefSeq) = preferido
- GCA_ (GenBank) = secundario

4. Fecha de liberación

- más reciente gana

El mejor ensamblaje se elige por:

```
(RefSeq score, Level score, RefSeq namespace, Release date)
```

(2) Descarga por HTTPS con verificación MD5

Función: `download_reference_genome(...)`

El módulo:

- Descarga `md5checksums.txt` del ensamblaje.
- Extrae todos los MD5 esperados.
- Descarga:
 - `_genomic.fna.gz`
 - `_genomic.gff.gz`

Y opcionalmente:

- `_protein.faa.gz`
- `_cds_from_genomic.fna.gz`
- `_genomic.gbff.gz`
- Verifica MD5 si `check_md5=True`.

Es **idempotente**:

→ NO descarga otra vez si el archivo ya existe y es válido.

(3) Escribe metadata de trazabilidad

Crea:

```
ASSEMBLY_PROVENANCE.txt
```

Con información exacta del ensamblaje seleccionado:

- Accession
- Organism
- Assembly name
- Status
- RefSeq category
- FTP usado
- Fecha

Esto es esencial para revisión por pares y reproducibilidad.

(4) Función principal reusable

```
fetch_reference_package(...)
```

Agrupar:

- selección del ensamblaje
- descarga reproducible
- validación de integridad
- escritura de metadatos

Devuelve:

```
(meta, fasta_path, gff_path)
```

(5) Modo CLI

Puedes usarlo desde consola:

```
python descarga_genoma_referencia.py -q "Klebsiella sp. AQSCr" -s AQSCr -o  
genome/
```

BLOQUE 1 — QC de lecturas crudas (FASTQ sin recortar)

1.1 `run_qc_raw.sh` (**FastQC + MultiQC en FASTQ crudos**)

Rol en el pipeline

- Evalúa la **calidad de las lecturas crudas** antes de cualquier recorte.
- Genera reportes por muestra (FastQC) y un resumen conjunto (MultiQC).
- Sirve para:
 - detectar adaptadores,

- ver distribución de calidad por base,
- revisar longitud de lecturas, GC, etc.

Flujo interno típico

1. Asume estructura de directorios ya montada

- Los FASTQ crudos están en:

```
data/fastq/*.fastq
```

- Crea (si no existe) el directorio de QC:

```
data/qc/
```

2. Corre FastQC sobre cada FASTQ

- Recorre `data/fastq` y para cada archivo:

```
fastqc -o data/qc data/fastq/SRRXXXXXX_1.fastq  
fastqc -o data/qc data/fastq/SRRXXXXXX_2.fastq
```

- Resultado por archivo:

- `data/qc/SRRXXXXXX_1_fastqc.html`
- `data/qc/SRRXXXXXX_1_fastqc.zip`
- (y lo mismo para `_2`).

3. Resumen global con MultiQC

- Una vez terminados los FastQC, ejecuta:

```
multiqc data/qc -o data/qc
```

- Genera:

- `data/qc/multiqc_report.html`
- archivos de soporte en `data/qc/multiqc_data/`

Output de este bloque

- Carpeta de QC para crudos:

```
data/qc/  
├─ SRR*_fastqc.html  
├─ SRR*_fastqc.zip  
└─ multiqc_report.html
```

```
└─ multiqc_data/...
```

- **Uso práctico:**
 - Revisar el `multiqc_report.html` para decidir parámetros de Trimmomatic (adaptadores, recortes, etc.).

BLOQUE 2 — Trimming + QC de lecturas recortadas

2.1 `run_trimmomatic.sh` (recorte de adaptadores y baja calidad)

Rol en el pipeline

- Limpia las lecturas:
 - elimina adaptadores (archivo `TruSeq3-PE.fa`),
 - recorta calidad baja en extremos,
 - elimina lecturas muy cortas.
- Genera FASTQ recortados listos para el alineamiento.

Flujo interno típico

1. Asume:

- FASTQ crudos en `data/fastq/`
- archivo de adaptadores:

```
data/trimmomatic/TruSeq3-PE.fa # symlink a tu archivo real
```

- Directorio de salida:

```
data/trimmed/
```

2. Para cada par SRR:

Llama a `trimmomatic PE` más o menos así:

```
trimmomatic PE \  
  data/fastq/SRR12998916_1.fastq \  
  data/fastq/SRR12998916_2.fastq \  
  data/trimmed/SRR12998916_1.trimmed.fastq \  
  data/trimmed/SRR12998916_1.unpaired.fastq \  
  data/trimmed/SRR12998916_2.trimmed.fastq \  
  data/trimmed/SRR12998916_2.unpaired.fastq
```

```
data/trimmed/SRR12998916_2.trimmed.fastq \  
data/trimmed/SRR12998916_2.unpaired.fastq \  
ILLUMINACLIP:data/trimmomatic/TruSeq3-PE.fa:2:30:10 \  
LEADING:3 TRAILING:3 SLIDINGWINDOW:4:15 MINLEN:36
```

- `.trimmed.fastq` = lecturas pareadas que sobrevivieron al recorte.
- `.unpaired.fastq` = lecturas huérfanas (una de las dos quedó muy mala).

3. Estructura resultante:

```
data/trimmed/  
├─ SRR12998916_1.trimmed.fastq  
├─ SRR12998916_1.unpaired.fastq  
├─ SRR12998916_2.trimmed.fastq  
├─ SRR12998916_2.unpaired.fastq  
...
```

Para el alineamiento usamos sólo los archivos `*.trimmed.fastq` pareados.

2.2 `run_qc_trimmed.sh` (FastQC + MultiQC en FASTQ recortados)

Rol en el pipeline

- Repite el QC pero ahora sobre los **FASTQ ya recortados**.
- Permite verificar si:
 - se eliminaron adaptadores,
 - mejoró la calidad promedio,
 - la longitud de lecturas es aceptable.

Flujo interno

1. Entrada:

- `data/trimmed/*trimmed.fastq`

2. Salida:

- Reportes en `data/qc_trimmed/`:
 - `SRR*_trimmed_fastqc.html`
 - `multiqc_report.html`

3. Comandos típicos:

```
fastqc -o data/qc_trimmed data/trimmed/SRRXXXXXX_1.trimmed.fastq
...
multiqc data/qc_trimmed -o data/qc_trimmed
```

BLOQUE 3 — Alineamiento y métricas de mapeo

3.1 `run_index_genome.sh` (indexar genoma para Bowtie2)

Rol en el pipeline

- Construye el índice Bowtie2 a partir del FASTA genómico ref.
- Este índice se usa luego en el alineamiento.

Flujo interno

1. Asume que tienes:

```
data/genome/GCF_008452795.1_ASM845279v1_genomic.fna
```

2. Corre algo como:

```
bowtie2-build \  
  data/genome/GCF_008452795.1_ASM845279v1_genomic.fna \  
  data/genome/bowtie2_idx/kleb
```

3. Genera archivos de índice:

```
data/genome/bowtie2_idx/  
├─ kleb.1.bt2  
├─ kleb.2.bt2  
├─ kleb.3.bt2  
├─ kleb.4.bt2  
├─ kleb.rev.1.bt2  
└─ kleb.rev.2.bt2
```

3.2 `run_bowtie2.sh` (alineamiento lecturas → genoma)

Rol en el pipeline

- Alinea las lecturas pareadas recortadas contra el genoma de referencia.
- Convierte SAM → BAM → BAM ordenado e indexado.
- Guarda logs de Bowtie2 para revisar tasas de mapeo.

Flujo interno

Para cada SRR:

1. Alineamiento con Bowtie2:

```
bowtie2 \  
-x data/genome/bowtie2_idx/kleb \  
-1 data/trimmed/SRRXXXXXX_1.trimmed.fastq \  
-2 data/trimmed/SRRXXXXXX_2.trimmed.fastq \  
-S data/alignments/SRRXXXXXX.sam \  
2> data/alignments/SRRXXXXXX.log
```

- El .log contiene el famoso:

```
99.72% overall alignment rate
```

2. Conversión SAM → BAM y ordenamiento:

```
samtools sort \  
-o data/alignments/SRRXXXXXX.sorted.bam \  
data/alignments/SRRXXXXXX.sam  
  
samtools index data/alignments/SRRXXXXXX.sorted.bam
```

3. Salida final por muestra:

```
data/alignments/  
├─ SRR12998916.sam  
├─ SRR12998916.sorted.bam  
├─ SRR12998916.sorted.bam.bai  
├─ SRR12998916.log  
...
```

3.3 run_flagstat.sh (métricas de mapeo con samtools)

Rol en el pipeline

- Resume estadísticas clave de mapeo por muestra:
 - número de lecturas,
 - porcentaje mapeado,
 - correctamente pareadas,
 - singletons, etc.
- Sirve como QC del alineamiento.

Flujo interno

Para cada `.sorted.bam` :

```
samtools flagstat data/alignments/SRRXXXXXX.sorted.bam \  
> data/alignments/stats/SRRXXXXXX.flagstat.txt
```

Ejemplo de salida:

```
11782308 + 0 in total  
11749248 + 0 mapped (99.72% : N/A)  
93746 + 0 properly paired (0.80% : N/A)  
...
```

- Ayuda a detectar problemas de mapeo (tasas muy bajas, etc.).

BLOQUE 4 — Conteo de lecturas por gen

4.1 `run_featurecounts.sh` (conteo de reads por gen con **featureCounts**)

Rol en el pipeline

- Toma los BAM alineados y el archivo GFF de anotación.
- Cuenta cuántas lecturas caen en cada **CDS / gen**.
- Produce la tabla de conteos brutos por gen y por muestra.

Flujo interno

1. Entrada:

- BAMs ordenados:

```
data/alignments/*sorted.bam
```

- Anotaciones:

```
data/genome/GCF_008452795.1_ASM845279v1_genomic.gff
```

2. Llamada a featureCounts (versión final que usaste):

```
featureCounts \  
-T 8 \  
-p -B -C \  
-t CDS \  
-g locus_tag \  
-a data/genome/GCF_008452795.1_ASM845279v1_genomic.gff \  
-o data/counts/counts_final.txt \  
data/alignments/SRR12998916.sorted.bam \  
... \  
data/alignments/SRR12998921.sorted.bam
```

Donde:

- `t CDS` → usa las features tipo CDS del GFF.
- `g locus_tag` → agrupa lecturas por `locus_tag` (ej. `BHE81_RS08845`).
- `p` → lecturas pareadas.
- `B -C` → filtros adicionales para pares válidos.

3. Salida principal:

- `data/counts/counts_final.txt`

Estructura:

```
# Program: featureCounts ...  
Geneid Chr Start End Strand Length SRR12998916 SRR12998917 ...  
BHE81_RS25410 NZ_MJDM01000011.1 1 109 + 109 0 0 0 0 0 0  
BHE81_RS25415 NZ_MJDM01000011.1 125 565 + 441 10 2 6 16 8 4  
...
```

Y el resumen:

- `data/counts/counts_final.txt.summary`
-

4.2 `prepare_dea_inputs.py` (conectar metadatos SRA + conteos → archivos DEA)

Rol en el pipeline

- Usa la tabla de conteos `counts_final.txt` y la tabla de metadatos `SraRunTable.csv`.
- Genera dos archivos clave para DEA:
 1. `data/metadata/design_matrix.csv`
→ diseño experimental (muestra, condición)
 2. `data/counts/counts_for_DEA.csv`
→ matriz de conteos limpia y ordenada para PyDESeq2.

Flujo interno (versión actual del script)

1. Entrada:

- `data/metadata/SraRunTable.csv`
- `data/counts/counts_final.txt`
- columna de condición (ej. `"Sample_Name"` o como se definió; el script normaliza a `control/treated`).

2. Pasos internos:

- Carga `SraRunTable.csv` y verifica que exista la columna `Run`.
- Elige/usa una columna de condición (p. ej. `Sample_Name`) y la normaliza:
 - Muestras en LB → `control`
 - Muestras en LB + Cr(VI) → `treated`
- Construye una tabla:

```
sample,condition
SRR12998916,control
SRR12998917,control
SRR12998918,control
SRR12998919,treated
SRR12998920,treated
SRR12998921,treated
```

- Guarda esto en:

```
data/metadata/design_matrix.csv
```

- Carga `counts_final.txt`:
 - Limpia nombres de columnas (`data/alignments/...sorted.bam` → `SRR12998916` , etc.).
 - Se queda con:
 - columna `Geneid`
 - columnas de SRR que aparezcan en el diseño.
 - Ordena las columnas según el orden de `design_matrix.csv` .
- Guarda la matriz final de conteos para DEA:

```
data/counts/counts_for_DEA.csv
```

Ejemplo:

```
Geneid,SRR12998916,SRR12998917,SRR12998918,SRR12998919,SRR12998920,SRR12998921
BHE81_RS25410,0,0,0,0,0,0
BHE81_RS25415,10,2,6,16,8,4
...
```

3. Checks de robustez:

- Verifica que todos los conteos sean enteros.
- Verifica que las muestras del diseño existen en la matriz de conteos.
- Imprime mensajes útiles si falta algo.

BLOQUE 5 — Análisis de expresión diferencial (DEA) + visualizaciones

5.1 `run_pydeseq2.py` (DEA con PyDESeq2, todo en Python)

Rol en el pipeline

- Implementa un análisis tipo DESeq2 pero en Python (`pydeseq2`), usando:
 - `counts_for_DEA.csv`
 - `design_matrix.csv`
- Calcula:

- normalización por tamaño de biblioteca,
- dispersión,
- test de Wald,
- p-valores y padj,
- conteos normalizados.
- Resume resultados y genera archivos para plots posteriores.

Flujo interno (versión actual que ya corriste)

1. Carga de datos:

- Lee:

```
data/counts/counts_for_DEA.csv
data/metadata/design_matrix.csv
```

- Extrae la lista de muestras del diseño:

```
muestras_diseño = ["SRR12998916", ..., "SRR12998921"]
```

- Reorganiza `counts_for_DEA` para que:
 - `Geneid` sea índice.
 - columnas = muestras en ese orden.
- Transpone para PyDESeq2 (muestras como filas, genes como columnas).

2. Preparar metadata:

- Usa `design_matrix.csv`, pone `sample` como índice y lo alinea con `counts_para_deseq`.
- Cuenta cuántas muestras hay de cada condición:

```
control: 3
treated: 3
```

3. Crear objeto `DeseqDataSet` :

- Sintaxis moderna:

```
dds = DeseqDataSet(
    counts=counts_para_deseq.astype(int),
    metadata=design_df,
    design="~ condition",
)
```

4. Ejecutar DESeq2 completo:

```
dds.deseq2()
```

Internamente hace:

- cálculo de **size factors** (normalización),
- estimación de **dispersión**,
- ajuste de modelo,
- cálculo de Cook's distance y outliers.

5. Estadística de DE con DeseqStats :

```
stats = DeseqStats(dds, contrast=("condition", "treated", "control"))
stats.run_wald_test()
resultados = stats.results_df
```

El contraste es: **treated vs control**.

6. Guardar resultados:

```
data/dea/deseq2_results.csv
```

Columnas típicas:

- baseMean
- log2FoldChange
- lfcSE
- stat
- pvalue
- padj

7. Obtener conteos normalizados:

- Busca en `dds.layers['normed_counts']` (o equivalentes).
- Si no hay, los calcula manualmente con los size factors.
- Los guarda en:

```
data/dea/normalized_counts.csv
```

- Imprime estadísticas:
 - forma (genes × muestras),
 - rango,
 - mediana.

8. Resumen final en consola:

- Top 10 genes más significativos (padj más bajo).
- Número total de genes analizados.
- Número y porcentaje de DEGs (padj < 0.05).
- Cuántos con $|\log_2FC| > 2$.
- Mensaje final:

```
ANÁLISIS COMPLETADO EXITOSAMENTE
```

5.2 plot_volcano.py (volcano plot de DEGs)

Rol en el pipeline

- Representa visualmente:
 - en X: `log2FoldChange`
 - en Y: `log10(padj)`
- Permite ver rápidamente:
 - genes con cambios grandes (FC alto),
 - genes con alta significancia (padj bajo),
 - separación entre sobrerregulados y subregulados.

Flujo interno típico

1. Lee:

```
data/dea/deseq2_results.csv
```

2. Filtra filas sin `log2FoldChange` o `padj`.

3. Calcula:

```
resultados["neg_log10_padj"] = -np.log10(resultados["padj"])
```

4. Define umbrales, por ejemplo:

- `padj < 0.05`
- `|\log_2FC| > 1` o `> 2`

5. Usa **matplotlib** para graficar:

- puntos en gris = no significativos,

- puntos en color (rojo/azul) = DEGs (significativos),
- líneas verticales/horizontales con los umbrales.

6. Guarda la figura en:

```
figures/volcano_plot.png
```

5.3 `plot_heatmap_top_genes.py` (heatmap de los top 50 genes DE)

Rol en el pipeline

- Visualiza los patrones de expresión de los genes más significativos.
- Permite ver clusters de genes, y cómo se agrupan las muestras por condición.

Flujo interno típico

1. Lee:

```
data/dea/deseq2_results.csv
data/dea/normalized_counts.csv
data/metadata/design_matrix.csv
```

2. Selecciona los **top N genes** (ej. 50) con menor `padj`.
3. Extrae de `normalized_counts` las filas correspondientes a esos genes.
4. Aplica transformaciones:
 - $\log_2(x+1)$ o z-score por gen para hacer comparables las escalas.
5. Crea un heatmap (o clustermap) con seaborn + matplotlib:
 - columnas = muestras, coloreadas por condición (`control` vs `treated`),
 - filas = genes ordenados por significancia o clustering.
6. Guarda la figura:

```
figures/heatmap_top50_genes.png
```

BLOQUE 6 — Gráficas EDA y análisis exploratorio

6.1 `plot_eda_counts.py` (análisis exploratorio de datos de expresión)

Rol en el pipeline

- Realiza **análisis exploratorio** de los conteos normalizados
- Visualiza la calidad y estructura de los datos antes del DEA
- Identifica patrones, outliers y agrupamientos naturales

Flujo interno

1. Boxplots de expresión por muestra

- Visualiza distribución de `log2(counts+1)` por muestra
- Permite identificar diferencias en dispersión entre muestras
- Detecta outliers técnicos

2. Density plots (KDE) por muestra

- Muestra distribución de densidad de expresión
- Compara perfiles de expresión entre condiciones
- Identifica sesgos en la distribución

3. Análisis de componentes principales (PCA)

- PC1 vs PC2 coloreado por condición
- Evalúa separación natural entre grupos experimentales
- Identifica batch effects o agrupamientos no esperados

4. Clustering jerárquico / dendrograma

- Muestra relaciones de similitud entre muestras
- Corroborar agrupamiento por condición experimental
- Identifica muestras mal agrupadas

5. Salidas gráficas

- `figures/eda/boxplot_expression.png`
- `figures/eda/density_plots.png`
- `figures/eda/pca_plot.png`
- `figures/eda/clustering_dendrogram.png`

6.2 `SEABORN.ipynb` (análisis exploratorio de datos de expresión)

Análisis EDA avanzado con Seaborn

```
# Ejecutar notebook de análisis Seaborn
jupyter notebook seaborn_analysis.ipynb
```

Salidas generadas:

- figures/eda/boxplot_violin_condition.png - Comparación por condición
- figures/eda/scatter_replicates.png - Relación entre réplicas
- figures/eda/expression_histograms/ - Distribuciones por muestra
- figures/eda/pairplot_samples.png - Análisis multivariado

RESULTADOS VISUALES COMPLETOS

Con este bloque adicional, el pipeline ahora incluye:

Visualización Básica + Avanzada

- **Volcano plot** - Significancia estadística
- **Heatmap** - Patrones de expresión
- **PCA** - Separación de grupos
- **Clustering** - Relaciones entre muestras
- **Density plots** - Distribuciones globales
- **Boxplots/Violinplots** - Comparación por condición
- **Scatter plots** - Correlación entre muestras
- **Histograms** - Distribución individual
- **Pairplots** - Análisis multivariado

Análisis Integral

- **Estadístico**: DEGs con p-valores y fold-changes
- **Funcional**: Anotación biológica completa
- **Visual**: Gráficas para publicación
- **Exploratorio**: Entendimiento profundo de datos

BLOQUE 7 — Anotación funcional de genes diferencialmente expresados

7.1 Anotación funcional completa

7.1.1 run_eggnog_from_proteins.py - Anotación con EggNOG-mapper

bash

python src/run_eggnog_from_proteins.py

- Anotación funcional completa de proteínas
- Asignación de términos GO, KEGG, COG
- Base para toda la interpretación biológica posterior

7.1.2 run_functional_annotation.py - Anotación integrada

bash

python src/run_functional_annotation.py

- Integra todas las anotaciones funcionales
- Procesa resultados de EggNOG, KEGG, COG
- Genera resumen unificado de anotaciones

7.1.3 run_cog_analysis.py - Análisis de categorías COG

bash

python src/run_cog_analysis.py

- Clasificación en Clusters of Orthologous Groups
- Distribución funcional de genes DE
- Identificación de categorías enriquecidas

7.1.4 run_kegg_enrichment_improved.py - Enriquecimiento de rutas

bash

python src/run_kegg_enrichment_improved.py

- Identifica pathways KEGG significativamente enriquecidos
- Análisis estadístico de rutas metabólicas
- Visualización de rutas alteradas

7.2 Análisis exploratorio y visualización

7.2.1 run_pca_analysis.py - Análisis de componentes principales

bash

```
python src/run_pca_analysis.py
```

- PCA de muestras por condición
- Visualización de agrupamientos naturales
- Detección de outliers y batch effects

7.2.2 run_clustering_analysis.py - Clustering jerárquico

```
bash
```

```
python src/run_clustering_analysis.py
```

- Dendrograma de similitud entre muestras
- Corroborar agrupamiento por condición experimental
- Identificación de relaciones entre réplicas

7.2.3 run_density_plots.py - Gráficas de densidad

```
bash
```

```
python src/run_density_plots.py
```

- Distribución de densidad de expresión
- Comparación de perfiles entre condiciones
- Identificación de sesgos en distribución

Resumen ordenado de scripts

```
# 0) Descarga SRA + genoma
python src/Pipeline_2.0.py

# 1) QC crudo
bash src/run_qc_raw.sh

# 2) Trimming + QC trimmed
bash src/run_trimmomatic.sh
bash src/run_qc_trimmed.sh

# 3) Index + alineamiento + flagstat
bash src/run_index_genome.sh
bash src/run_bowtie2.sh
bash src/run_flagstat.sh

# 4) Conteos + preparación DEA
bash src/run_featurecounts.sh
```

```
python src/prepare_dea_inputs.py --condition-column <columna>
```

```
# 5) DEA + figuras
```

```
python src/run_pydeseq2.py
```

```
python src/plot_volcano.py
```

```
python src/plot_heatmap_top_genes.py
```

```
# 6) Gráficas EDA
```

```
python src/plot_eda_counts.py
```

```
SEABORN.ipynb
```

```
# 7) Anotación funcional
```

```
python src/run_eggnog_from_proteins.py
```

```
python src/run_functional_annotation.py
```

```
python src/run_cog_analysis.py
```

```
python src/run_kegg_enrichment_improved.py
```

Cómo correr todo el pipeline de principio a fin

Pipeline completo de RNA-seq para *Klebsiella* sp. AqSCr (CrVI)

Desde descarga → QC → alineamiento → conteos → DEA → figuras

Esta guía asume que estás ubicado en la raíz del proyecto y que tienes instalados:

- **FastQC, MultiQC**
- **Trimmomatic**
- **Bowtie2**
- **samtools**
- **subread/featureCounts**
- **Python 3 + pydeseq2 + pandas + seaborn + matplotlib**
- **NCBI SRA Toolkit** (`prefetch` , `fasterq-dump`)
- **Visualización**
- **Anotación funcional**

0. Descarga automatizada de datos y genoma de referencia

Este paso obtiene **todos los SRR** del proyecto SRA y descarga el **genoma de referencia óptimo** desde NCBI Assembly.

```
python src/Pipeline_2.0.py
```

Resultados esperados:

- data/raw_sra/ → archivos .sra
 - data/fastq/ → FASTQ pareados
 - data/genome/ → FASTA + GFF (.fna , .gff)
-

1. Control de calidad (QC) – Lecturas crudas

Corre FastQC/MultiQC sobre los FASTQ sin procesar.

```
bash src/run_qc_raw.sh
```

Resultados:

- Reportes en data/qc/
 - multiqc_report.html para revisar calidad general.
-

2. Trimming y QC post-trimming

2.1 Recorte (Trimmomatic)

```
bash src/run_trimmomatic.sh
```

Salida:

- FASTQ recortados en data/trimmed/

2.2 QC en FASTQ recortados

```
bash src/run_qc_trimmed.sh
```

Salida:

- data/qc_trimmed/multiqc_report.html
-

3. Alineamiento contra el genoma de referencia

3.1 Construir índice Bowtie2

```
bash src/run_index_genome.sh
```

Salida:

- Índice en data/genome/bowtie2_idx/

3.2 Alineamiento

```
bash src/run_bowtie2.sh
```

Salida:

- .sam, .sorted.bam, .bam.bai en data/alignments/
- Log con “overall alignment rate”.

3.3 Métricas con flagstat

```
bash src/run_flagstat.sh
```

Salida:

- data/alignments/stats/*.flagstat.txt
-

4. Conteo de lecturas por gen

4.1 featureCounts – Generar matriz de conteos

```
bash src/run_featurecounts.sh
```

Salida:

- data/counts/counts_final.txt
- data/counts/counts_final.txt.summary

4.2 Preparar matrices para DEA (ordenar muestras según metadatos)

```
python src/prepare_dea_inputs.py --condition-column Sample_Name
```

Salida:

- data/metadata/design_matrix.csv
 - data/counts/counts_for_DEA.csv
-

5. Análisis de expresión diferencial (DEA)

```
python src/run_pydeseq2.py
```

Salida:

- Resultados DESeq2: data/dea/deseq2_results.csv
 - Conteos normalizados: data/dea/normalized_counts.csv
 - Resumen detallado impreso en consola.
-

6. Visualizaciones (volcán, heatmap)

Volcano plot

```
python src/plot_volcano.py
```

Salida:

- `figures/volcano_plot.png`

Heatmap top 50 DEGs

```
python src/plot_heatmap_top_genes.py
```

Salida:

- `figures/heatmap_top50_genes.png`

Gráficas complementarias de exploración

```
# Ejecutar notebook de análisis Seaborn  
jupyter notebook seaborn_analysis.ipynb
```

Salidas generadas:

- `figures/eda/boxplot_violin_condition.png` - Comparación por condición
- `figures/eda/scatter_replicates.png` - Relación entre réplicas
- `figures/eda/expression_histograms/` - Distribuciones por muestra
- `figures/eda/pairplot_samples.png` - Análisis multivariado

7. Anotación funcional e interpretación biológica

7.1 Anotación funcional de genes diferencialmente expresados

bash

```
python src/run_functional_annotation.py
```

Salida:

- `data/annotation/functional_annotation_summary.csv` - Anotaciones completas
- `data/annotation/DEGs_biological_interpretation.txt` - Resumen biológico
- Identificación de genes de respuesta a Cr(VI) (transportadores, enzimas detoxificantes)

7.2 Análisis de enriquecimiento de rutas

bash

```
python src/run_pathway_enrichment.py
```

Salida:

- `data/annotation/kegg_pathway_enrichment.csv` - Rutas KEGG enriquecidas
 - `data/annotation/go_enrichment_analysis.csv` - Términos GO significativos
 - `figures/pathway_enrichment_plots/` - Visualizaciones de rutas
-

7.3 Análisis exploratorio de datos (EDA)

7.1 Gráficas EDA de conteos normalizados

bash

python src/plot_eda_counts.py

Salida:

- `figures/eda/boxplot_expression.png` - Distribución por muestra
 - `figures/eda/density_plots.png` - Densidades de expresión
 - `figures/eda/pca_plot.png` - PCA por condición
 - `figures/eda/clustering_dendrogram.png` - Dendrograma de similitud
-

Pipeline completo

Una vez ejecutados todos los pasos, habrás producido:

Resultados de análisis

- FASTQ crudos + FASTQ recortados
- Reportes de QC completos (crudos y recortados)
- Lecturas alineadas al genoma con métricas de mapeo
- Matrices de conteos brutos y normalizados
- Resultados estadísticos de expresión diferencial (DEGs)

Interpretación biológica COMPLETA

- Anotación funcional completa de todos los genes DE
- Identificación de pathways KEGG enriquecidos
- Clasificación COG de funciones proteicas
- Términos GO de procesos biológicos alterados

- Contexto biológico de la respuesta a Cr(VI)

Visualizaciones integrales

- Volcano plot de DEGs
- Heatmap de top 50 genes diferenciales
- PCA de separación entre condiciones
- Clustering jerárquico de muestras
- Gráficas de densidad de expresión
- Visualizaciones de enriquecimiento funcional

Reproducibilidad y trazabilidad

- Metadatos completos del genoma (ASSEMBLY_PROVENANCE.txt)
- Diseño experimental documentado
- Scripts modulares y reusables
- Estructura de proyecto organizada y estandarizada

Todo de forma **reproducible y modular**, con un script por etapa.
